

CM3070 Final Project Report

ContextID: A Context-Aware Digital Identity Management System

Student: Zhe Wang

Date: September 2026

Live deployment: <https://www.contextid.app/>

Project Template: 7 CM3035 Advanced Web Design, 7.1 Project Idea 1 — *Identity and profile management API*

Chapter 1: Introduction

1.1 Motivation and Problem Statement

People hold many roles at once: a software engineer, a musician, a parent, a graduate of a particular university. In ordinary life this plurality is unremarkable — a person presents differently at a conference than at a family dinner, without needing to reconcile the two. Online, few platforms preserve that separation. LinkedIn places work history and education on a single page; a personal social feed mixes colleagues and family; a single email address quietly links accounts a person may have preferred to keep apart. The academic literature has a name for the resulting failure mode — *context collapse*, the flattening of previously distinct audiences into one undifferentiated space (Marwick and boyd, 2011) — and it is a useful frame for the rest of this report, because it treats the problem as a structural property of platform design rather than a matter of individual carelessness.

The consequences are concrete. Someone at a conference may want to share contact details without disclosing a personal phone number. A person with both a Chinese and an English name may find that a single "full name" field cannot represent either correctly. Someone who shares a phone number with a new contact may later want to limit or withdraw that access, only to discover that most systems provide no mechanism to do so once information has left their control. These are not edge cases; they are ordinary experiences for anyone who moves between social or professional groups. Existing tools — LinkedIn, digital business-card applications, link-in-bio pages — each address a fragment of the problem, but none combine separate contexts, fine-grained sharing control, and genuine revocability in one system aimed at non-technical users.

1.2 Project Concept

ContextID is a web-based identity management platform built on a simple premise: one user account may hold several identity cards, each scoped to a context such as Professional, Academic, Personal, Family, or a user-defined label. Each card carries its own structured name — a mandatory given name plus an optional family name, additional given name, and secondary family name — so that naming conventions outside the Western first/last pattern, including ones with no family name at all, can be represented correctly rather than forced into it. A card may also carry a display name, courtesy title, email, phone

number, location, short description, photograph, and a validity period, so that an affiliation can expire naturally instead of remaining visible after it has ceased to apply.

The system's central feature is sharing by link rather than by copy. The owner selects a visual template and generates a unique URL for a specific card; the recipient opens that link and sees a clean, unauthenticated public page. The owner can revoke the link at any time from a dashboard listing every link created, active or not, giving them ongoing control over each disclosure rather than a single irreversible act of sharing a complete profile.

The system targets people with no particular technical background. Creating an identity is a short guided flow: choose a context, then edit the card directly while it already looks the way a recipient will see it. Sharing takes one click and copies the link automatically. There is no password to remember — sign-in is by Google, GitHub, or a one-time email link.

1.3 Project Template

This project is based on Project Template 7, CM3035 Advanced Web Design, Project Idea 7.1 — Identity and profile management API. Course materials on secure account management and REST API design (CM3035 Advanced Web Design, University of London) directly informed the security and API-design decisions discussed in §2.9 and §2.10. The brief asks for a web application built around a REST API, where users can manage their identity and profile information and have control over what information they share with other people. ContextID follows this idea by providing a full-stack application with a Fastify REST API, a PostgreSQL database using Prisma, and a Next.js client that communicates with the API. The project also covers the deployment process and requirements analysis.

1.4 Technical Approach

The system is a TypeScript monorepo managed with Turborepo and pnpm: a Next.js 16 frontend using the App Router, and a Fastify v5 REST API, both reading and writing through a single Prisma schema and client against a shared PostgreSQL database. Profile photographs are stored in Amazon S3.

Authentication is handled by Auth.js v5, supporting Google OAuth, GitHub OAuth, and passwordless email sign-in through Resend, with sessions persisted server-side in the database rather than trusted from a signed cookie alone.

Deployment is automated with GitHub Actions and Terraform: pushing a version tag builds and pushes Docker images for both applications to Amazon ECR, applies pending database migrations, and deploys to AWS ECS behind a resolved custom domain, using GitHub's OIDC federation so that no long-lived AWS credentials are stored in the repository.

1.5 Target Users and Value Proposition

ContextID is designed for professionals, academics, multilingual users, and anyone who often shares their contact information but does not want everyone to have the same full and permanent profile. The main idea is to give users more control over what they share. They can choose what information to show, how the profile looks, and how long it stays available, and they can change these settings later. A printed business card is usually a one-time action that cannot be changed or taken back. With ContextID, the owner keeps control of their digital card for as long as they want, while recipients can still save the information to their contacts or scan a QR code.

1.6 Report Structure

Chapter 2 extends the literature review, in response to feedback that it lacked depth, with self-sovereign identity and usable security as further contrasts. Chapter 3 revises the design chapter to add the architecture, data-model, and interface diagrams the same feedback identified as missing, plus a risk-and-contingency assessment. Chapter 4 describes the implementation, including work completed since the draft — a design-system migration, input-validation hardening, contact export, and authentication-flow fixes. Chapter 5 evaluates critically: two security vulnerabilities and two accessibility defects found and fixed, a route-level test suite and CI gate built where none existed, and an explicit, evidence-weighted verdict against this project's objectives rather than a list of gaps softened by reassurance — including the one gap, independent usability evidence, that remains genuinely unresolved. Chapter 6 concludes with prioritised future work.

Chapter 2: Literature Review

This chapter revises and extends the literature review from the draft report. The preliminary submission was assessed as having a limited review; the response taken here is not only to add citation density but to use each source to make a specific, checkable claim about ContextID's design, including where the system falls short of what the source would recommend, rather than treating the literature as background colour for a design already fixed.

2.1 Context Collapse and Contextual Integrity

Nissenbaum's (2004) theory of contextual integrity gives a clearer way to understand this problem. She argues that privacy is not simply about keeping information secret or controlling data. Instead, it depends on whether information is shared in a way that fits the situation where it was originally given. For example, sharing your job information at a conference and showing the same information on a dating profile uses the same data, but the two situations have very different expectations. Privacy can therefore be broken even when someone has permission to access the information, if the information is being shared in an inappropriate context. This is one of the main reasons for ContextID's design, where users create separate identity cards for different contexts instead of having one profile and adding visibility settings afterward.

It is also important to explain what this design cannot prevent. Nissenbaum's theory focuses on whether information is shared appropriately within a context. It does not cover what someone does with the information after they receive it. ContextID's features, such as hard-to-guess links, revocation, and expiry, cannot stop someone from sharing a card outside its intended context. Revoking a link prevents future access through that link, but it cannot remove a screenshot or a copy that someone has already made. This is a real limitation of the system, rather than a problem that has been completely solved, and it is discussed again in the evaluation in Chapter 5.

2.2 Personal Names Are More Structured Than Software Assumes

Most software stores a name as two fields, first and last. Ishida's (2011) W3C Internationalization guidance documents at length how poorly this generalises: in China, Japan, Korea, and Hungary the family name conventionally precedes the given name; Spanish and Portuguese names typically carry two

family names, one from each parent; Icelandic names have no family name in the Western sense at all; and automated parsing of a three-token name cannot reliably determine, without cultural context, whether it represents two given names and one family name or the reverse. Ishida's methodological point — that name fields should be designed around what a form needs to *display and address a person correctly*, not around a fixed Western template — is the one ContextID adopts directly, storing given name, optional family name, and optional additional given and secondary family name components separately, alongside a distinct, freely editable display name per identity. The family name field was not optional in an earlier version of the schema, a contradiction of this section's own Icelandic example that went unnoticed until it was corrected in §4.6.

This separation matters beyond formatting. ISO/IEC 24760-1:2019, the identity-management terminology standard, defines an identity as a set of attributes related to an entity *within a particular context*, explicitly permitting the same entity to hold multiple, simultaneously valid identities. Read alongside Ishida, this supports treating the display name as a first-class, context-specific attribute rather than a derived string: a legal name is comparatively invariant, but the name a person wishes to be *addressed by* is not, and conflating the two — as most systems do — is itself a design error rather than an internationalisation gap.

2.3 Digital Identity Theory: Cameron's Laws of Identity

Cameron's (2005) *Laws of Identity*, written while he was Microsoft's Chief Identity Architect, remains one of the clearest attempts to state what a trustworthy digital identity system should do. Two of the seven laws bear directly on ContextID and are worth measuring the system against rather than simply citing.

The Law of Minimal Disclosure states that a system should reveal the smallest amount of identifying information a purpose requires, and for no longer than necessary. ContextID only partly satisfies this. A share link is scoped to one identity card rather than a user's entire account, a real improvement over a single monolithic profile, but the card itself is shared as one indivisible unit: a recipient who needs only an email address also receives the phone number, location, and description, because ContextID does not yet support disclosing individual fields separately. This is flagged as an explicit, unresolved gap and is the top functional item in the future-work list in Chapter 6.

The Law of Directed Identity distinguishes *omnidirectional* identifiers — public and easily discoverable, such as a company's DNS name — from *unidirectional* identifiers, intended for one specific relying party and not meant to be linkable across contexts. ContextID's share tokens function as unidirectional identifiers: each is generated independently per recipient and is not derived from the identity's own database identifier, so two recipients holding different links to the same card cannot use the tokens themselves to infer they refer to the same person. This is a genuine, if partial, alignment with Cameron's model, and gives stronger privacy than most consumer sharing tools, which typically expose a single stable public profile URL to every visitor.

2.4 Self-Sovereign Identity: A Contrasting Paradigm

A different research tradition — self-sovereign identity (SSI) — rejects the server-mediated model ContextID adopts, and is worth engaging with directly because it represents the strongest alternative design philosophy available, not merely a related idea. Allen's (2016) widely cited formulation of SSI principles argues that identity should be held and controlled by the individual, portable across services, and independent of any single administrative authority, typically realised through decentralised identifiers

and cryptographically signed, holder-presented credentials. The W3C's Verifiable Credentials Data Model (W3C, 2022) formalises this as a three-party model — issuer, holder, verifier — in which a credential, once issued and cryptographically signed, can be verified offline by any party without contacting the issuer at presentation time.

This is a direct and instructive contrast with ContextID's approach. ContextID is deliberately centralised and *online-verified*: a recipient's browser must contact ContextID's server on every visit to a share link, because that is precisely the mechanism that makes revocation possible. A self-issued verifiable credential, once presented and its signature checked, cannot be revoked in the same immediate sense unless the verifier separately checks a revocation list at presentation time — an obligation the SSI model places on the verifier, not the holder, and one that is frequently omitted in practice. ContextID trades the SSI model's portability and offline verifiability for a property SSI does not straightforwardly offer: an owner-initiated "off switch" that a recipient's client cannot silently ignore. Neither design is strictly superior; they optimise for different threat models, and ContextID's choice should be read as a deliberate one rather than an unawareness of the alternative.

2.5 Prior Art and the Gap

Several existing tools address fragments of the same problem. LinkedIn presents a single professional identity with no mechanism for showing a different profile to different viewers. Virtual business-card applications such as HiHello and Blinq support link-based sharing but, in their free tiers, offer only one card per account and no link-level revocation once a card has been sent. Link-in-bio tools such as Linktree and About.me aggregate a single public identity rather than supporting multiple, separately controlled ones. None of the four combine multiple context-scoped identities, individually revocable and expirable share links, structured non-Western name fields, and zero-account access for the recipient.

Tool	Multiple contexts	Access control	Link revocation	No recipient account	Structured names
LinkedIn	No	Coarse	No	No	No
HiHello / Blinq	Partial	No	No	Yes	No
About.me / Linktree	No	No	No	Yes	No
ContextID	Yes	Yes	Yes	Yes	Yes

This table should not be read as ContextID being unambiguously superior — each competitor optimises for something ContextID does not attempt, such as Linktree's aggregation of many external links on one page, or LinkedIn's network effects. The narrower and more defensible claim is that none of them are built around Nissenbaum's flow-appropriateness principle as a first-class design constraint, and ContextID is.

2.6 Privacy Regulation and Privacy by Design

The GDPR (Regulation (EU) 2016/679) grants a right to erasure (Article 17) and requires privacy to be considered by design and by default (Article 25). Cavoukian's (2009) earlier Privacy by Design framework argued for exactly this: that privacy should be built into a system's architecture rather than bolted on as policy afterward. ContextID's revocation mechanism follows this principle structurally — a share is

revoked by setting a **revokedAt** timestamp that the resolution endpoint checks on every request, so the capacity to withdraw access exists from the moment a share is created, not as a manually operated exception process.

This claim needs the same qualification given in §2.1: revocation invalidates the *token*, not the information a recipient has already viewed, copied, or memorised. Both contextual integrity and the GDPR's erasure right describe an ideal of complete withdrawal that no link-based sharing system, this one included, can fully deliver once data has left the system. ContextID's genuine contribution is making the technical act of withdrawal immediate and free of charge — a real improvement over an emailed vCard or a printed card, neither of which can be revoked at all — without overstating what that improvement achieves. Token revocation and Article 17 are distinct guarantees, though: revoking a share withdraws one link's access to data that still exists, while erasure removes the data itself. ContextID implements both — a share-level revocation as described above, and an account-level deletion (§4.9) that satisfies Article 17 directly.

2.7 REST APIs for Identity Management

Fielding's (2000) dissertation introduced Representational State Transfer as an architectural style built around addressable resources, a small uniform set of methods, and a statelessness constraint: no request may rely on context left over on the server from a *previous* request in the same interaction, so each request must carry, itself, everything needed to authorise and process it. This is a narrower claim than "no server-side state exists" — a server may still hold state about resources, including session records, provided a client never needs to have been "remembered" across requests to be served correctly. This fits an identity system well for the read side in particular: identities, contexts, and shares are naturally distinct resources with their own URLs and lifecycles, and it is exactly this property that lets an unauthenticated recipient open a public share link with no prior interaction with the system at all. HTTP content negotiation (MDN, 2024), via headers such as **Accept-Language**, could in principle let a REST identity API return a name in a specific script or language where more than one representation exists; ContextID does not yet implement this, and it remains future work (Chapter 6).

2.8 Usable Security and the Limits of Technical Controls

A recurring theme of this chapter is that a technically correct control is not automatically an effective one if it is not legible to the person relying on it. Whitten and Tygar's (1999) classic usability evaluation of PGP 5.0 found that participants with genuine technical competence still failed, at alarming rates, to use a cryptographically sound tool correctly, because the interface did not make the security model comprehensible at the moment a decision had to be made. The lesson generalises beyond encryption: a revocation feature that exists in the API but is hard to find, or a destructive action whose confirmation is easy to dismiss without reading, provides less real protection than its technical correctness would suggest. This motivates treating interface clarity around consequential actions — discussed concretely in the usability evaluation in §5.5 — as a security-adjacent property of the system, not merely a cosmetic one.

2.9 Web API Security

A system that stores personal information and deliberately exposes part of it publicly must treat access control as a design property, not an implementation detail. The OWASP Application Security Verification Standard (OWASP, 2021) recommends at least 64 bits of entropy for security-sensitive tokens such as session identifiers and password-reset links, since shorter tokens become guessable at scale, and

requires authorisation to be re-checked on every request that changes data, against the specific resource being changed. This addresses what OWASP's API Security Top 10 (OWASP, 2023) lists as the most common API security risk: broken object-level authorisation.

ContextID's share tokens use nine bytes of cryptographically secure randomness, giving 72 bits of entropy and exceeding the 64-bit baseline. The second requirement — authorisation checked per resource, per request — proved harder to satisfy consistently, as Chapter 5 describes in detail: a real gap existed and was found and fixed during evaluation. This illustrates precisely why OWASP frames the check as an ongoing discipline rather than a one-time design decision — it is straightforward to implement correctly in one route file and unintentionally omit from another as an API grows.

2.10 Authentication and Session Management

Cameron's Law of Minimal Disclosure (§2.3) extends naturally to authentication itself: a system should not require a credential it does not need to operate securely. ContextID uses OAuth's authorisation-code flow for Google and GitHub sign-in and a single-use, time-limited email link for passwordless sign-in, so the application never receives, stores, or is capable of leaking a password, because none is ever created — consistent with the wider move, driven substantially by the prevalence of credential-stuffing attacks against reused passwords (OWASP, 2021), away from long-lived shared secrets and toward delegated or possession-based authentication. Sessions are stored server-side and re-validated against the database on every request rather than trusted purely from a signed token's claims. This is a deliberate departure from strict self-descriptiveness — a stateless-in-the-purest-sense design would trust a signed JWT's claims without a database lookup — traded away because it bounds the usable lifetime of a stolen session token independently of whether the token itself is cryptographically compromised: revoking a session means deleting a row, not waiting for a token to expire. §2.7 discusses the narrower, per-request sense in which the API's authenticated routes remain stateless despite this lookup.

2.11 Synthesis

Across sources that use different vocabularies, this review converges on one principle: information should be shared only as widely, for as long, and with as many people as its context genuinely requires. Nissenbaum (2004) calls this appropriate flow; Cameron (2005) calls it minimal disclosure; the GDPR frames it as data minimisation and erasure; Whitten and Tygar (1999) add the condition that a control only counts if people can actually use it correctly. Self-sovereign identity (§2.4) shows that even this principle admits more than one architecture: ContextID's centralised, revocable model and the SSI holder-controlled model both aim at user control over disclosure, but trade off portability against immediacy of revocation in opposite directions.

None of the consumer tools surveyed in §2.5 pursues this principle system-wide. ContextID builds it in at the data-model level — separate identities per context, independently revocable shares, unlinkable per-recipient tokens — rather than adding it later as configuration. The review is equally clear about the limits of this approach: ContextID cannot un-share information a recipient has already seen or copied, cannot yet disclose a card at field granularity, and its revocation control is only as effective as the interface that presents it — a claim tested directly in the usability evaluation of Chapter 5.

Chapter 3: Design

This chapter adds the architecture, data-model, and interface diagrams that feedback on the preliminary submission identified as missing, and closes with the risk-and-contingency assessment the same feedback requested.

3.1 Requirements

Functional Requirements

ID	Requirement
FR1	A user can register and sign in using Google OAuth, GitHub OAuth, or an email magic link
FR2	A user can create an identity card assigned to a named context
FR3	System-provided contexts (Personal, Work, Family, Social) are pre-seeded and available to all users
FR4	A user can create custom contexts in addition to system contexts
FR5	Each identity stores an optional courtesy title, given name, optional family name, optional additional given name, optional secondary family name, display name, optional email, phone number, location, description, and a validity date range
FR6	A user can upload a profile photograph per identity; a deterministic placeholder is shown otherwise
FR7	A user can preview an identity in every available visual template before sharing it
FR8	A user can generate a share link for any identity in any one template, with an optional expiry date
FR9	A share link is revocable by the owner at any time and preserves an audit record after revocation
FR10	An expired or revoked share link returns a distinguishable error (410 Gone) to the viewer
FR11	The public share page renders the identity using the template selected at share time, and is accessible without authentication
FR12	A user can update any field of an existing identity
FR13	A user can delete an identity and all its associated shares
FR14	No user can read, modify, share, or delete another user's identity, context, or share record through any API route
FR15	A recipient of a shared identity can save it directly to their device's contacts (vCard) or open it by scanning a QR code, without needing an account
FR16	Every field with a length limit is enforced identically on the client and the server, so a request that bypasses the UI cannot store data the UI would have rejected
FR17	A user can permanently delete their own account, including every identity, context, and share link they own and any uploaded photos, after a typed confirmation step

FR14 remains stated explicitly, in response to the ownership-check gap fixed during the evaluation in Chapter 5; it was previously only an implicit, unguaranteed consequence of FR1. FR15, FR16, and FR17 are new, reflecting functional additions made since the draft report. FR5 now states family name as optional rather than mandatory, correcting a requirement that contradicted this report's own naming literature (§2.2, §4.6).

Non-Functional Requirements

Security: every operation on a specific resource requires the resource's owner to match the authenticated caller (FR14). Share tokens carry at least the OWASP-recommended 64 bits of entropy. Uploaded images are validated for MIME type and size.

Internationalisation: name fields accommodate non-Latin naming structures, including compound family names, multiple given names, and naming systems with no family name at all (§2.2, §4.6).

Usability: editing shows a live, direct-manipulation preview rather than a disconnected form, and no master–detail view is left silently empty when data exists (§3.4).

Accessibility: pages target WCAG 2.1 AA (W3C, 2018). An automated audit (§5.6) has since found and fixed two defects (an unlabelled control and several placeholder-only fields) and found, but not yet fixed, a systemic colour-contrast defect; no manual keyboard-navigation audit has been done.

Data protection: a user can exercise the GDPR Article 17 right to erasure (FR17) unassisted, with no support request required; §4.9 covers the mechanism.

3.2 System Architecture

ContextID is a pnpm monorepo managed by Turborepo:

```
CM3070-FINAL-PROJECT/
├── apps/
│   ├── web/           Next.js 16 (App Router) – port 3000
│   └── api/           Fastify v5 REST API – port 4000
├── packages/
│   └── database/      Prisma schema, migrations, seed, shared Prisma client
```

Figure 1 shows which component talks to which. Server Components read from PostgreSQL directly through the shared Prisma client, avoiding a network hop; client-side mutations go through a same-origin proxy route handler (`/api/proxy/[...path]`) that reads the `HttpOnly` Auth.js session cookie server-side and re-issues the request to Fastify with the session token as a `Bearer` header. This keeps the token out of reach of browser JavaScript and lets the API enforce authentication and ownership independently of the calling client — relevant directly to the security analysis in Chapter 5.

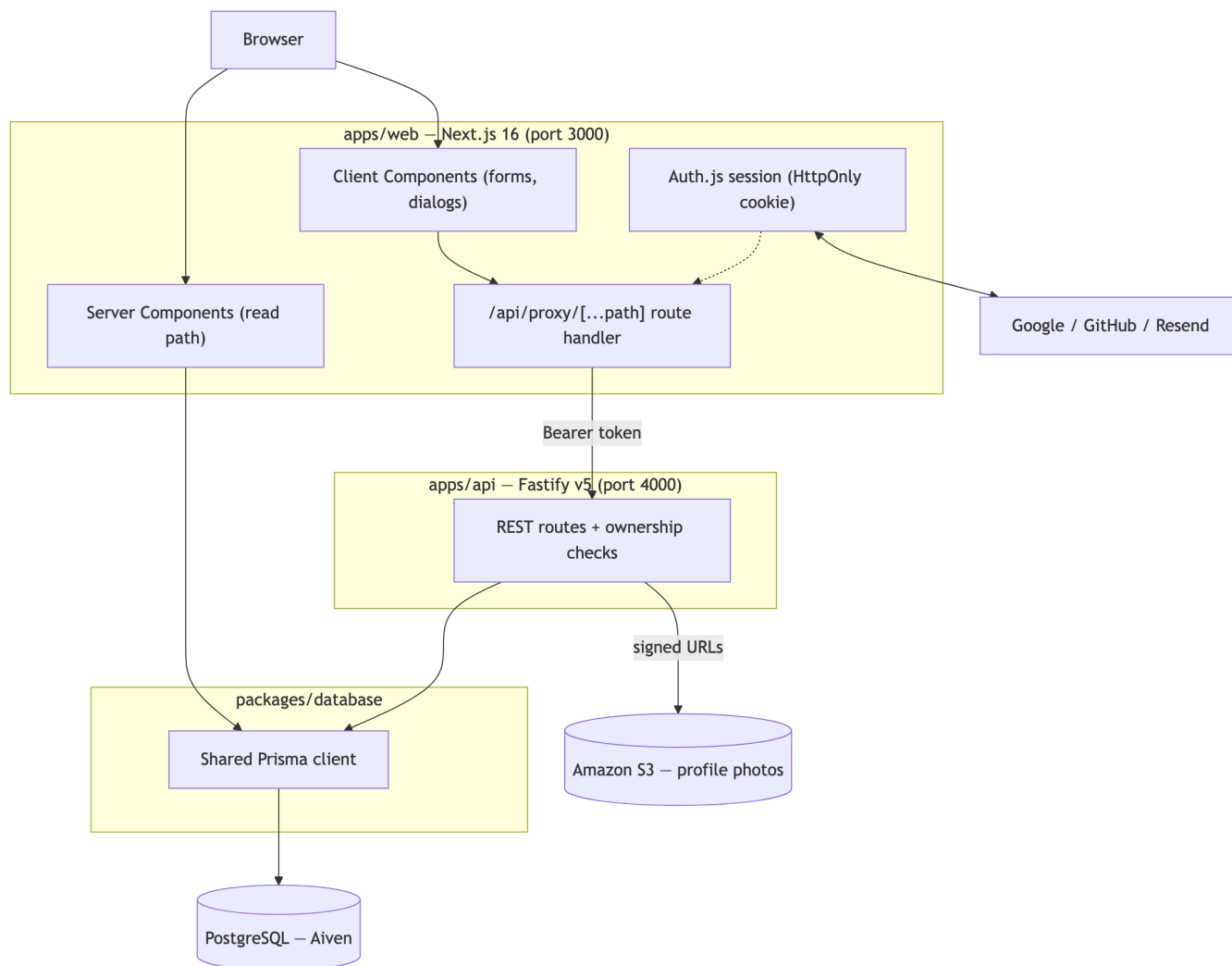


Figure 1 — System architecture: read and write paths, and the trust boundary between the browser and the session token.

3.3 Data Model

The domain schema has five core tables, extended since the preliminary report with fields the card needed to read as complete rather than a bare name:

- **User** — account record, managed by Auth.js
- **IdentityContext** — named context, nullable **userId** (null for system contexts). A unique constraint on (**userId**, **name**) prevents duplicate custom context names per user
- **Identity** — the identity card, including structured name fields, image, email, description, validity range, courtesy title, location, and phone. A unique constraint on (**userId**, **contextId**) enforces one identity per context per user
- **IdentityShare** — the share record, carrying both *which* identity is shared and *which visual template* it should render in, decoupling the owner's presentation choice from any single recipient's view
- **Account / Session / Authenticator / VerificationToken** — Auth.js adapter tables

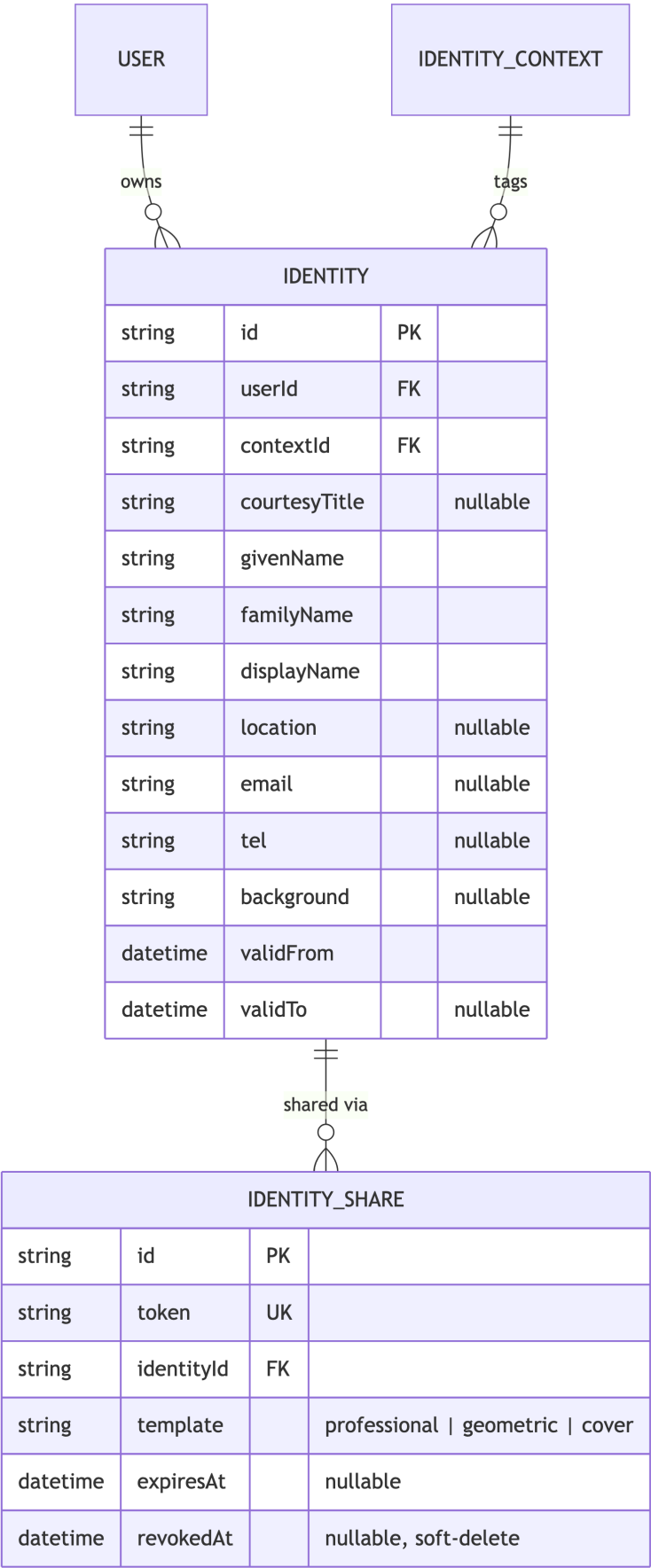


Figure 2 — Entity-relationship diagram of the core domain schema.

3.4 Interface and Navigation Design

Figure 3 sets out the intended path through the interface. Two properties are deliberate rather than incidental: the recipient-facing path (from the public share page onward) requires no account or prior state, consistent with the stateless, resource-oriented design in §2.7; and the owner-facing path treats the identity card itself as the primary editing surface (§3.5), not a separate form.

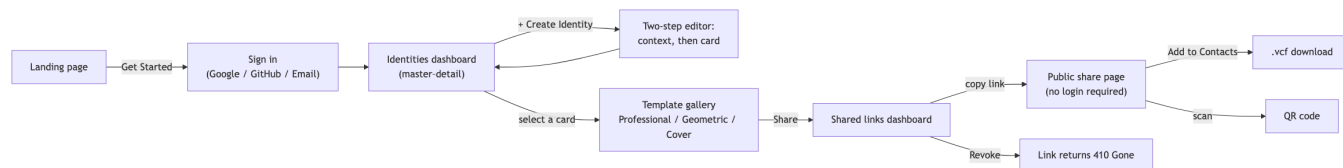


Figure 3 — End-to-end interface and navigation flow, owner and recipient paths.

One usability consequence of this flow is addressed directly: on first arrival at either dashboard, nothing was selected and the detail panel was simply empty — a dead end, not an invitation to explore. Both views now default the panel to the first item on the current page when nothing is explicitly selected, so a blank pane is never shown when data exists — a small change motivated by heuristic evaluation (§5.5) and consistent with Nielsen's (1994) *recognition rather than recall* heuristic.

3.5 Key Design Decisions

Direct-manipulation identity editing. The prototype's three-step wizard (context, plain name form, separate preview) is collapsed to two steps: choose a context, then edit the card itself directly. Every recipient-visible field is entered on the live rendered card; legal name fields, which exist for correctness rather than presentation, sit in a plainer section beneath it — directly serving the usability requirement in §3.1 that a user should never have to imagine what they are creating.

Per-share template selection. Three visually distinct templates — Professional, Geometric, and Cover — are chosen at share time and stored on the **IdentityShare** record, not the identity. Cover also supports a curated background image, owner-chosen or defaulted deterministically per identity so a card never looks unfinished. The same identity can thus be shared plainly to one audience and expressively to another without duplicating the record — extending Chapter 2's contextual-separation principle into presentation, not only data.

A consistent, accessible component system. Hand-rolled overlay `<div>`s for every modal, styled ad hoc, were replaced with a small set of shared primitives (dialog, alert-dialog, dropdown menu, select) built on an unstyled, accessibility-focused library, so focus trapping, Escape-to-dismiss, and ARIA semantics are provided once rather than re-implemented — inconsistently — per component. This changes what the interface *guarantees*, not only its appearance; §4.5 and §5.6 discuss what it does and does not establish about accessibility.

Overflow menus for card-level actions. Presenting every action as an always-visible button does not scale as a list grows. Destructive and secondary actions are now grouped behind one overflow menu, trading one extra click for less clutter — a conventional, and on the evidence of §5.5, acceptable trade-off.

Master–detail navigation and soft-deleted shares. The dashboard remains a persistent-sidebar, paginated master list with a detail panel for the selected identity's template gallery, scaling better than an inline preview per row. A revoked share still returns **410 Gone** rather than being deleted, so "never existed" and "deliberately withdrawn" stay distinguishable — unchanged since the preliminary design.

3.6 Infrastructure: Plan Versus What Was Built

The preliminary report proposed ECS Fargate behind an Application Load Balancer, AWS Amplify for the frontend, and a new Amazon RDS instance inside a custom VPC. What was actually built, in Terraform, differs in three deliberate respects: both applications deploy as managed ECS "Express" gateway services rather than Fargate-plus-Amplify, halving the number of deployment mechanisms at the cost of some configuration control; the account's default VPC is used rather than a custom-provisioned one, a reasonable simplification here but a real trade-off against the isolation a production multi-tenant deployment would need; and the database remains the existing Aiven instance, since migrating live, accumulated data mid-project carries risk for no functional benefit. Since the draft, deployment also resolves a registered custom domain via Terraform-managed DNS, needed for Google's OAuth client to accept the callback URL as a stable origin. Deployment triggers on a pushed version tag, not every merge to `main`; Chapter 5 evaluates the gap between this and a pull-request-gated check.

3.7 Risks and Contingency

The preliminary work plan was assessed as unrealistic and lacking any account of risk. This section states the risks judged material for the remainder of the project, and the contingency adopted for each.

Risk	Likelihood	Impact	Mitigation / Contingency
Insufficient time to build a full CI pipeline and test coverage	High	Medium	Prioritise ownership-check regression tests (§5.2) first, for the highest security value; keep a CI gate as the top future-work item rather than dropping it silently (§5.3, §6.3)
No independent usability evidence obtainable in time	High	Medium	Substitute heuristic evaluation (Nielsen, 1994), explicitly labelled as weaker than a participant study rather than presented as equivalent (§5.5)
No accessibility audit tool or session available in time	Medium	Medium	Adopt accessibility-oriented component primitives as a structural mitigation (§3.5, §4.5); state plainly this is not a substitute for an audit (§5.6)
Solo development, no second reviewer	Certain	Medium	Treat type-checking and the test suite as a non-optional gate; use disciplined, adversarial self-review (§5.2) in place of a reviewer who does not exist
Fast-moving frontend dependencies (Next.js 16, new accessibility library)	Medium	Low	Pin exact lockfile versions; re-verify build and type-check after every dependency-affecting change
Field-level disclosure (§2.3, §6.3) not completed in time	High	Low	Scope explicitly as future work; the current all-or-nothing model is a stated limitation, not a silent gap

Chapter 4: Implementation

4.1 Overview

The implemented system now covers every functional requirement in §3.1, including substantial work completed since the draft report: a migration to an accessible, consistent component system across every dialog and form in the application; input-validation hardening at both the client and the API; a recipient-facing contact-export and QR-sharing feature; a full account-deletion path satisfying the GDPR right to erasure; and two fixes to the authentication flow that were found while building it, not planned in advance. This chapter covers the mechanisms judged most significant for each: ownership verification, the share-token lifecycle, per-template sharing, the component-system migration, input validation, contact export, account deletion, and the authentication-flow fixes, each with the code and reasoning behind it.

4.2 Ownership Verification (IDOR Protection)

Every route that reads, updates, deletes, or shares a specific resource must establish two facts before touching the database: that the caller is authenticated at all, and that the caller *owns the specific resource addressed*. The first is centralised in a Fastify `preHandler` hook registered on every protected route group, which validates the session's Bearer token against the `Session` table and attaches the resulting `userId` to the request. The second cannot be centralised the same way, because "ownership" means something different per resource type, and is checked against the specific row being acted on inside each handler:

```
// apps/api/src/routes/identities.ts – PATCH /identities/:id
const existing = await prisma.identity.findUnique({
  where: { id: request.params.id },
  select: { userId: true },
});

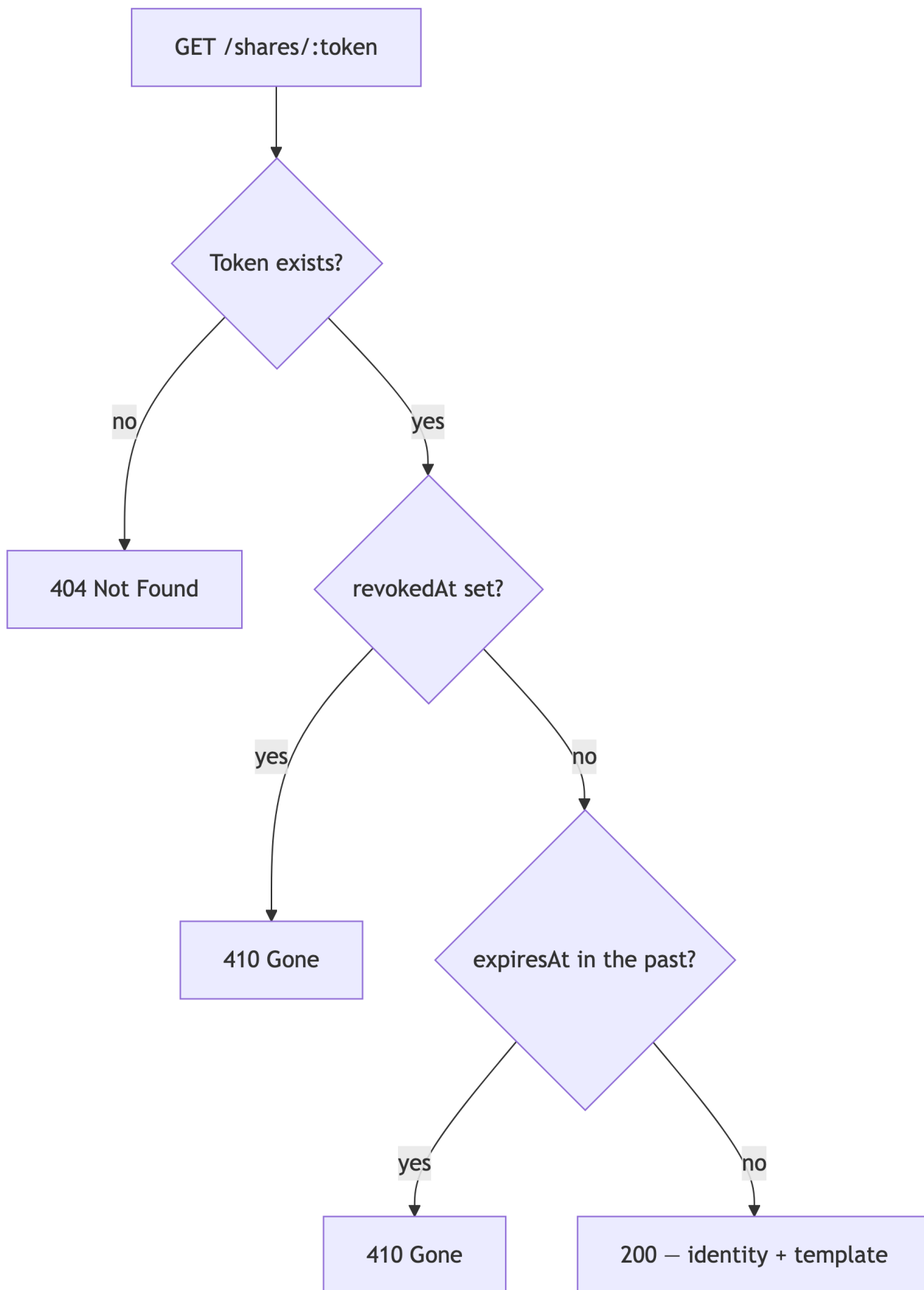
if (!existing) {
  return reply.status(404).send({ error: "Identity not found" });
}

if (existing.userId !== request.userId) {
  return reply.status(403).send({ error: "Forbidden" });
}
```

This pattern — fetch just the owner column, compare against `request.userId` (never against anything the client supplied), reject before mutating — is applied identically across `GET/PATCH/DELETE` on `/identities/:id` and `/identity-contexts/:id`, and the list endpoints `/users/:id/identities` and `/users/:id/identity-contexts`. `POST /identities` takes `userId` from `request.userId` rather than the request body, closing the possibility of a caller creating a record under another account. §5.2 describes in detail why this consistency did not exist until a dedicated audit, and how the gap was found.

4.3 Share Tokens and the Three-State Resolution

Share tokens are generated with `randomBytes(9).toString("base64url")` — 72 bits of entropy, above the 64-bit OWASP baseline (§2.9). The public resolution endpoint evaluates a token through an ordered check, returning a distinct status per outcome so a recipient can distinguish "never existed" from "deliberately withdrawn":



This is verified by four unit tests in `apps/api/src/routes/shares.test.ts`, each asserting one branch of the diagram above against a mocked Prisma client.

4.4 Per-Template Sharing

The public page resolves and renders whichever template a specific link was created with, read from a single registry that both the owner-facing preview gallery and the recipient-facing page consume:

```
// apps/web/components/name-card-templates/index.ts
export const TEMPLATES = {
  professional: { label: "Professional", Component: ProfessionalTemplate,
Card: ProfessionalCard },
  geometric:    { label: "Geometric",      Component: GeometricTemplate,
Card: GeometricCard },
  cover:        { label: "Cover",          Component: CoverTemplate,
Card: CoverCard },
} as const;
```

The Cover template additionally reads a background image, either the identity's own choice or a deterministic default:

```
// apps/web/lib/background-presets.ts
export function getIdentityBackgroundSrc(identity: { id: string;
background: string | null }): string {
  const chosen = BACKGROUND_PRESETS.find((preset) => preset.id ===
identity.background);
  if (chosen) return chosen.src;
  return BACKGROUND_PRESETS[hashString(identity.id) %
BACKGROUND_PRESETS.length]!.src;
}
```

Hashing the identity's own ID into a preset index, rather than always falling back to the same background, means every identity that has not yet chosen one still looks visually distinct in the owner's gallery — the same deterministic-variety pattern already used for the placeholder avatar colour, now reused for a second purpose rather than re-invented. This mapping is verified by four unit tests in `background-presets.test.ts`, covering both the explicit-choice and hashed-fallback branches.

4.5 Adopting an Accessible Component System

Every dialog and confirmation in the interface was originally a hand-rolled `<div>` acting as a modal: a fixed-position overlay, a click handler comparing `event.target === event.currentTarget` to detect a backdrop click, and no keyboard handling at all — pressing Escape did nothing, and focus was not constrained to the dialog's contents. This was replaced with a shared set of dialog, alert-dialog, dropdown-menu, and form-control primitives built on Base UI (MUI, 2024), an unstyled component library providing focus management and ARIA semantics as a property of the primitive rather than something each call site must reimplement.

Two real defects were found and fixed while carrying out this migration, and are reported here because they are the same kind of finding as the security audit in §5.2: a claim ("the dialog behaves correctly") that held only until it was actually tested. First, the library's own setup tooling defaulted the application to

a manually toggled dark-mode strategy, when every existing **dark:** utility class in the codebase — sidebar, buttons, cards, name-card templates — depended on the browser's **prefers-color-scheme** media query instead; left uncorrected, this would have silently frozen the entire application in light mode regardless of the visitor's system theme. Second, the new dialogs initially had no maximum height, so a long form (the identity editor, with more than a dozen fields) could grow taller than the viewport with no internal scroll, pushing its own footer buttons off-screen — caught only by deliberately shrinking the browser window during manual testing rather than by any automated check, which is itself evidence for the testing gap discussed in §5.3.

```
// components/identity-actions-menu.tsx — overflow menu replacing two
// separate buttons
<DropdownMenu>
  <DropdownMenuTrigger render={
    <Button variant="ghost" size="icon-sm" aria-label="Identity actions" />
  }>
    <MoreVertical />
  </DropdownMenuTrigger>
  <DropdownMenuContent align="end">
    <DropdownMenuItem onClick={() => setEditOpen(true)}><Pencil />
    </Edit></DropdownMenuItem>
    <DropdownMenuItem variant="destructive" onClick={() =>
      setDeleteOpen(true)}><Trash2 />Delete</DropdownMenuItem>
    </DropdownMenuContent>
  </DropdownMenu>
```

A destructive-action confirmation that was previously an inline "Sure? Yes / No" pair of text links, easy to miss or misclick, is now a proper alert dialog naming the specific identity or link being affected, with a clearly weighted destructive action — reported as a usability fix, with method and evidence, in §5.5.

4.6 Input Validation and Data-Integrity Hardening

No field on the identity record originally had a length limit, on either the client or the server. Investigating this surfaced a genuine, if narrow, database-level risk rather than only a UI-polish concern: every text field in the schema is a plain PostgreSQL **text** column with no practical size limit, except **IdentityContext.name**, which participates in a unique index (**@@unique([userId, name])**) — and PostgreSQL's btree index implementation errors on an indexed value large enough to exceed a page's row-size limit. Every field was therefore given an explicit, justified maximum (legal and display names at 100 characters, description at 200, location at 100, email at 254 per RFC 5321 (Klensin, 2008), phone at 30, context name at 50, specifically because of the index constraint above), enforced identically as a server-side check and as an HTML **maxlength** attribute:

```
// apps/api/src/routes/identities.ts
function validateFieldLengths(body: Partial<IdentityBody>): string | null {
  const nameFields: [string, string | undefined][] = [
    ["givenName", body.givenName], ["familyName", body.familyName],
    ["displayName", body.displayName],
  ];
  for (const [field, value] of nameFields) {
    if (value && value.length > NAME_MAX_LENGTH) {
```

```

        return `${field} must be at most ${NAME_MAX_LENGTH} characters`;
    }
}
// ...description, location, email, tel checked the same way
return null;
}

```

Enforcing the same limit twice, rather than only in the browser, matters specifically because every mutating route is reachable directly, and correctly requires no more than a valid session and resource ownership (§4.2) — a client-side-only limit would be trivially bypassed by any authenticated caller, which is exactly the caller the ownership checks in §4.2 are designed to still let through for their own resources.

A second, unrelated data-integrity gap was found in the same pass: `familyName` was a mandatory, non-nullable column, required by the API and by both the create and edit forms, despite §2.2 of this same report citing Ishida (2011) to argue that Icelandic names, among others, have no family name in the Western sense at all. The requirement was not a considered design decision; it was simply never revisited after the initial two-field prototype, and it directly contradicted a claim this report makes about its own data model. `familyName` is now nullable in the schema and optional at every layer that touches it — the API's create validation, both forms, and vCard generation (§4.7), whose `N` property correctly leaves the family-name component blank between semicolons when none exists, which is what RFC 6350 (Perreault, 2011) itself specifies for a name with a missing component, rather than requiring the user to fabricate a value to satisfy a database constraint. `givenName` remains mandatory, since no naming convention surveyed in §2.2 lacks a given name entirely.

4.7 Contact Export and QR Sharing

A recipient of a shared identity can now save it directly to their device's contacts, generating an RFC 6350-compliant (Perreault, 2011) vCard on demand:

```

// apps/web/lib/build-vcard.ts
export function buildVCard(identity: VCardIdentity): string {
    const fullName = identity.courtesyTitle ? `${identity.courtesyTitle}
    ${identity.displayName}` : identity.displayName;
    const lines = ["BEGIN:VCARD", "VERSION:3.0",

    `N:${escapeText(identity.familyName)};${escapeText(identity.givenName)};;;`,
    ,
    `FN:${escapeText(fullName)}]`;
    if (identity.email)
    lines.push(`EMAIL;TYPE=INTERNET:${escapeText(identity.email)}`);
    if (identity.tel)
    lines.push(`TEL;TYPE=CELL:${escapeText(identity.tel)}`);
    lines.push("END:VCARD");
    return lines.join("\r\n");
}

```

Escaping is applied per the vCard 3.0 text-value grammar (backslashes, semicolons, commas, and newlines) rather than passed through unescaped, since a description field containing a comma or

semicolon would otherwise corrupt the structure of the generated card for the receiving contacts application. This is verified by four unit tests covering the escaping rules and the presence or omission of optional fields. The public share page also renders a QR code encoding the share URL, so a card can be exchanged in person by camera scan as readily as by sending a link — both features directly realise the "printed business card" comparison made in §1.5, by giving a physical-style interaction (scan, save) to a link-based, revocable card.

4.8 Hardening the Passwordless Email Flow

Auth.js's default email sign-in flow signs the user in — and consumes the one-time token — on the first `GET` request to the callback URL. Email security scanners operated by mail providers (Microsoft Safe Links, Google Safe Browsing, and similar) pre-fetch links in incoming mail before a recipient opens the message, which silently consumes the token before the genuine click ever arrives, producing a confusing "link no longer valid" error with no user-visible cause. The email now links to ContextID's own confirmation page instead of the real callback URL directly; an automated pre-fetch of that intermediate page is inert, since only an actual click through the confirmation button reaches the real, token-consuming callback. The email itself was also given a branded HTML template, matching the application's own visual identity (logo mark, accent colour) rather than Auth.js's generic default, and a shared pending-state pattern (`useTransition`, one boolean covering all three sign-in methods) now disables the other two sign-in controls the instant one is submitted, closing a genuine race window in which a user could, for instance, click GitHub while a Google sign-in redirect was still in flight.

4.9 Account Deletion (GDPR Right to Erasure)

GDPR Article 17 gives a data subject the right to erasure, and nothing about ContextID's data (identity cards, share links, uploaded photos) falls under an exception that would let that right be refused. `DELETE /users/:id` implements it, guarded by the same self-only ownership check as every other route (§4.2), followed by a single `prisma.user.delete()`:

```
// apps/api/src/routes/users.ts
if (request.params.id !== request.userId) {
  return reply.status(403).send({ error: "Forbidden" });
}

await prisma.user.delete({ where: { id: request.params.id } });
await deleteUserS3Objects(app, request.params.id);
```

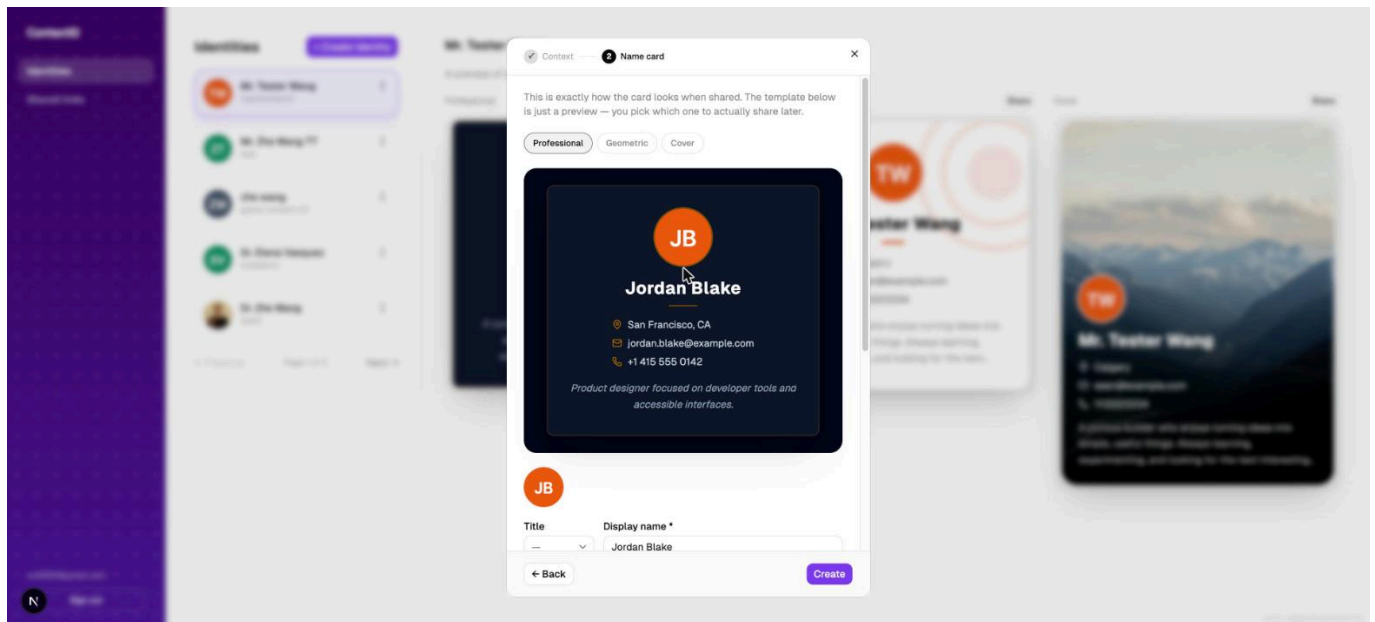
Every relational table cascades from `User` in the schema (`onDelete: Cascade` on `Account`, `Session`, `IdentityContext`, `Identity`, and — one hop further — `IdentityShare`), so this one call removes the account and everything under it in a single database transaction, atomically. Only after that commits does the handler best-effort delete the user's S3-stored photos, listing and batch-deleting every object under the `users/{id}/` prefix. This ordering is deliberate: the database delete is what actually removes personal data from a live, queryable system, so it happens unconditionally and first; if the S3 cleanup call then fails, the result is an orphaned object with no account left to belong to, logged for later reconciliation, rather than a half-deleted account. An asynchronous job queue was considered and rejected for this — the operation touches a handful of rows and at most a few photos, and

DeleteObjects batches up to 1000 keys in one call, so the added infrastructure (broker, worker, retry/dead-letter handling) would not be earning its cost at this scale.

On the client, deletion sits behind a stronger confirmation than the single "Delete?" alert used elsewhere (§4.5): the destructive action stays disabled until the user types their own account email into a field, and the dialog leads with an explicit, visually distinct "This cannot be undone" warning before listing what is being removed — proportionate friction for an action with no recovery path, where the identity-level confirmation (§4.5) is not. The control itself is tucked behind a small overflow menu next to the account email rather than sitting as a standing button in the sidebar, so it is reachable but not something a user brushes past on every visit.

4.10 Visual Representation

Two-step identity editor — every field is edited directly on the card that will be shared, not on a separate form:



Master–detail dashboard with the template gallery — the left column is a paginated list of the signed-in user's identities; selecting one renders every available template in the right-hand panel, each with its own Share control:

ContextID

Identities

Shared links

wp588@gmail.com

N Sign out

Identities

Create identity

TWMr. Tester WangnewContext3

ZTMr. Zhe Wang TTtest

ZWzhe wanggame-context-01

EVDr. Elena VasquezAcademic

Dr. Zhe Wangtest2

PreviousPage 1 of 3Next

Mr. Tester Wang

A preview of every name-card look currently available. More templates are on the way.

Professional

Share

Geometric

Share

Cover

Share

TW

Mr. Tester Wang

Calgary

sean@example.com

1112223334

A curious builder who enjoys turning ideas into simple, useful things. Always learning, experimenting, and looking for the next...

TW

Mr. Tester Wang

Calgary

sean@example.com

1112223334

A curious builder who enjoys turning ideas into simple, useful things. Always learning, experimenting, and looking for the next...

TW

Mr. Tester Wang

Calgary

sean@example.com

1112223334

A curious builder who enjoys turning ideas into simple, useful things. Always learning, experimenting, and looking for the next interesting...

v0.10 - 2026-08-29 16:20 UTC

22 / 32

Professional

Share



Mr. Tester Wang

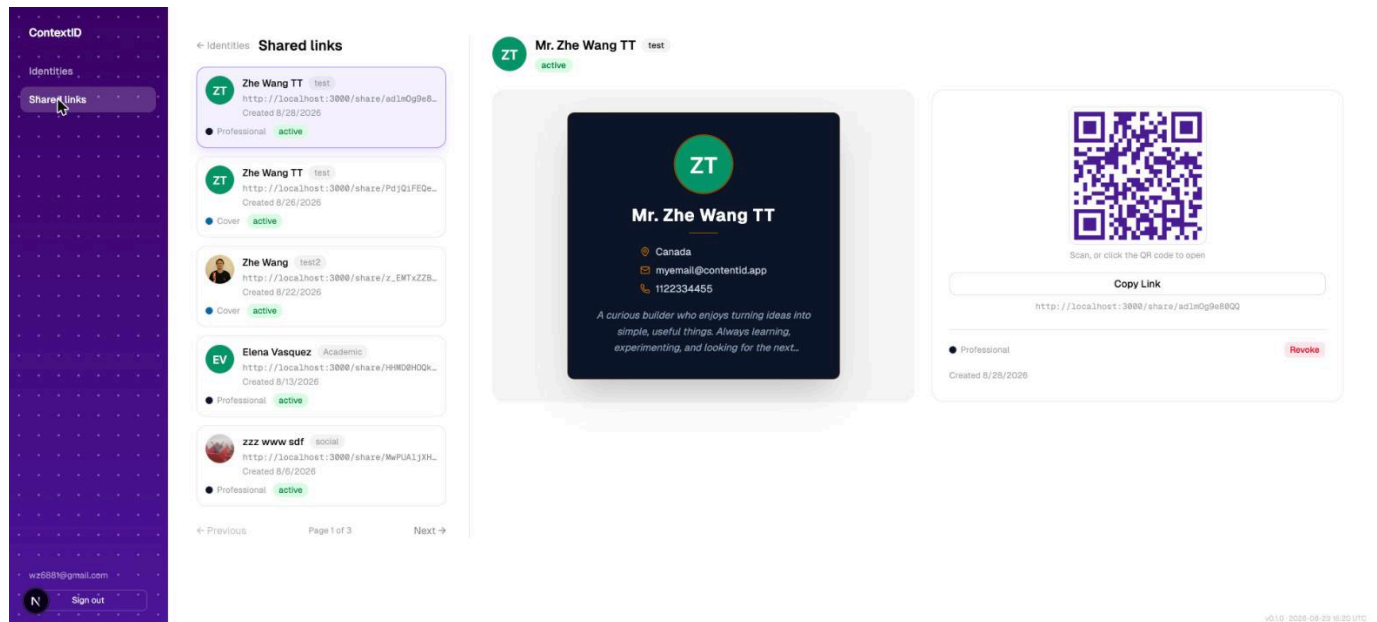
📍 Calgary

✉️ sean@example.com

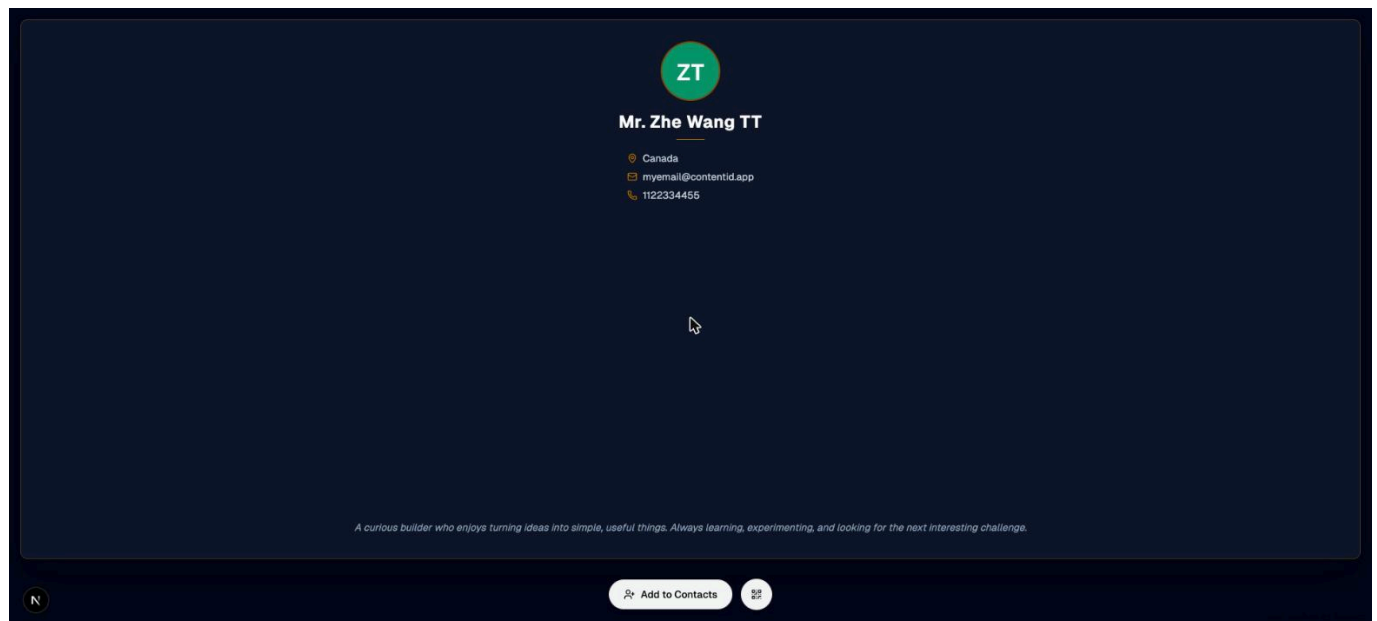
📞 1112223334

A curious builder who enjoys turning ideas into simple, useful things. Always learning, experimenting, and looking for the next...

Shared links dashboard — every link created for the signed-in user, across all identities and templates, with its current status:



Public share page — the unauthenticated recipient's view, rendered in the template chosen at share time:



Chapter 5: Evaluation

This chapter extends the evaluation from the draft report and is deliberately more critical than the prototype's evaluation was: it reports two genuine security vulnerabilities and two genuine accessibility defects, all discovered and fixed while preparing this revision; gives an honest, updated account of automated testing and CI, now real and passing rather than absent or partial; and states plainly what has still not been independently verified. In direct response to feedback that the preliminary evaluation rested mainly on self-testing and planned work, and that a later revision's analysis still leaned toward the positive rather than weighing evidence evenly, each subsection below states the specific measure used and what evidence it does and does not constitute, and §5.7 weighs that evidence against this project's objectives explicitly rather than concluding the project is nearly complete.

5.1 Functional Correctness

All functional requirements in §3.1 were exercised manually end-to-end against the running application: sign-in via each provider, identity creation and editing through the two-step editor (§3.5), the full share lifecycle across all four reachable states (active → 200, expired → 410, revoked → 410, non-existent → 404), rendering in each of the three templates, the vCard/QR recipient flow (§4.7), and account deletion (§4.9) — confirming the typed-confirmation gate blocks the action until the email matches, and that the account, its identities, its share links, and its S3 photos are all gone afterward and its share links immediately stop resolving. This confirms the features work as designed under normal use. It is not, by itself, regression protection, which §5.3 addresses critically.

5.2 Security: An Ownership-Check Audit, a Real Finding, and a Fix

Earlier reports claimed that ownership checks on every write made the system's protection against Insecure Direct Object Reference (IDOR) attacks impossible to bypass by calling the API directly. While preparing the draft report, that claim was checked systematically, route by route, against the actual code rather than re-asserted from memory — and found to be **only partially true**.

What was found. The share routes (create, list, revoke) correctly compared the resource's `userId` against `request.userId` on every handler. The identity and identity-context routes did not: `GET`, `PATCH`, and `DELETE` sat behind the global authentication check — proving the caller held a valid session — but never verified the caller owned *this specific* resource. Before the fix, any authenticated user could read, edit, or delete any other user's identity or context by supplying its identifier in the URL, and `POST /identities` trusted a `userId` field taken directly from the request body. This is precisely the broken-object-level-authorisation pattern discussed in §2.9, and it existed because the correct pattern had been applied to one route file but not propagated to the others as the API grew — exactly the failure mode an automated regression test, not present at the time, exists to catch.

Impact. Severity was bounded by resource identifiers being Prisma `cuid()` values, not sequentially guessable, so exploitation would have required a caller to already know or discover another user's identifier through some other channel. The vulnerability was real and a genuine defect, not a theoretical one, and was not observed to have been exploited.

The fix. Ownership checks matching the pattern already correct in the share routes (§4.2) were added to every affected route, and `POST /identities` was changed to take `userId` from the session rather than the request body. The fix was verified by a clean API type-check and a manual re-read of every affected handler — weaker verification than an automated regression test would provide, which is the subject of §5.3.

Token entropy, unaffected by the above, was reverified analytically: `randomBytes(9)` yields 72 bits, exceeding OWASP's 64-bit minimum; at a generous 10,000 requests per second, exhausting the token space remains computationally infeasible on the order of 10^{13} years. No rate limiting exists on the public share endpoint as a defence-in-depth measure; this gap, identified in the prototype report, remains open.

5.3 Testing and CI: From Absent, to Partial, to a Working Gate

The draft report stated plainly that no automated tests existed anywhere in the repository; a later revision added twelve unit tests but left the specific regression coverage this section had called for — route-level

ownership tests and a PR-gated CI workflow — still undone. Both are now built, in the same evaluation pass documented in §5.2, and this section reports what actually exists rather than what is still planned.

Tests. Thirty-six unit and route-level tests now exist and pass (`pnpm turbo run test`): the original twelve (share-resolution state machine, §4.3; vCard generation and escaping, §4.7; deterministic background-preset selection, §4.4) plus twenty-four new ones added specifically to close the gap identified above. The new tests assert, for `identities.ts`, `identityContexts.ts`, and the owner-facing share-lifecycle routes in `shares.ts` (create, list, revoke), that a second authenticated user receives 403 rather than another user's resource on every guarded route — the exact class of defect found and fixed in §5.2 — and that `POST /identities` attributes a new identity to the session, not to a `userId` supplied in the request body, directly regression-testing that specific fix. Writing these tests against the real route handlers, rather than only against the fixed code, surfaced one further, unrelated defect: a bare `GET /users/:id` endpoint in `index.ts` sat entirely outside the authenticated block, requiring no session at all and returning a user's full record, including email, to any caller who guessed or obtained their id. Nothing in the frontend called it; it has been removed rather than fixed, since the routes in `users.ts` already cover every legitimate use of that data.

CI. A new `ci.yml` workflow, independent of the existing tag-triggered deploy workflow and requiring no secrets, now runs on every pull request and push to `main`, executing `pnpm lint`, `pnpm check-types`, and `pnpm test` in sequence and failing the check if any of the three fails. Getting this to pass cleanly was not merely a matter of adding the workflow file: `pnpm lint` had never actually been run as a gate before, and doing so for the first time surfaced ten pre-existing warnings — three undeclared environment-variable dependencies in `turbo.json`, seven `catch (err: any)` blocks across the route files — that the project's `--max-warnings 0` policy had silently never been enforcing. These are now fixed (the `any` casts replaced with `instanceof Prisma.PrismaClientKnownRequestError` checks, the env vars declared), which matters beyond tidiness: a CI gate that is added but does not actually pass on the code it is meant to guard is not a gate, it is a workflow file, and verifying it goes green against a clean checkout — not merely that the YAML is syntactically valid — was treated as part of the deliverable, not a formality.

This closes the single highest-priority item identified in the draft and the previous revision of this report. What it does not do is retroactively verify that no other undiscovered gap exists of the same shape as the `GET /users/:id` finding above; the ownership-check pattern is now tested everywhere it is known to apply, not proven absent everywhere it could conceivably have been missed.

5.4 Infrastructure and Deployment: A Critical Look at the Deviation from Plan

§3.6 described three deliberate simplifications against the original infrastructure plan. Evaluated critically rather than merely reported: this was the right call for a solo, time-boxed project, reducing the number of independently configured AWS primitives from roughly six to two managed services, at a real cost — the project no longer demonstrates hands-on custom VPC and load-balancer configuration, part of the original technical ambition. The tag-triggered deploy pipeline is a similarly defensible choice, preventing an unreviewed merge from deploying automatically; it now complements, rather than substitutes for, the independent PR-gated CI check added in §5.3, which is the arrangement the draft report identified as missing.

5.5 Usability

No formal usability study has been conducted; the five-to-eight participant think-aloud study planned since the preliminary report has not started, and is reported here as not yet done rather than described in anticipation of results that do not exist. What follows instead is a critical developer walkthrough and heuristic evaluation against Nielsen's (1994) ten usability heuristics, a materially weaker form of evidence than a study with independent participants, and read as such.

Three concrete defects were found by this method and fixed, each reported with the specific heuristic it violated, as evidence of a repeatable method rather than a single anecdote. First, once a user has created one identity in each of the four system contexts, the context-selection step offered only "New context" with no explanation of why the system contexts had disappeared, violating *help users recognise, diagnose, and recover from errors* — a one-line explanatory message is planned before final submission and is not yet built. Second, a destructive action's confirmation was an inline "Sure? Yes / No" text pair, easy to activate accidentally and giving no visual weight to the irreversibility of the action, violating *error prevention*; this is fixed, replaced by a proper alert dialog naming the specific resource and giving the destructive action distinct, deliberate styling (§4.5). Third, the detail panel of both master-detail dashboards was silently empty on first arrival, violating *recognition rather than recall*; this is fixed by defaulting to the first available item (§3.4). The first defect remains open; the second and third are fixed and were re-verified manually after the fix.

5.6 Accessibility

The target set in the preliminary report was WCAG 2.1 Level AA (W3C, 2018), "verified by automated and manual review." That review had not been carried out at the draft stage; it has now been run, against the real running application rather than in the abstract, and it found genuine defects rather than confirming a clean bill of health.

Method. axe-core 4.10 was run in-browser against the sign-in screen, the identity dashboard, both steps of the create-identity editor, and the public share page, using a throwaway test account created directly in the database (the same technique as the project's own manual database-session testing) rather than a real OAuth or email login, so the same account could be exercised repeatedly without depending on a third-party provider.

What was found and fixed. Two defects were real and are now fixed: the courtesy-title dropdown in both the create and edit identity forms had no accessible name at all — a screen-reader user would hear only "combobox" with no indication of its purpose — and six free-text fields across the same two forms (display name, location, email, phone, description, and the four legal-name fields) relied on placeholder text as their only label, which is not a reliable accessible name and disappears the moment a value is entered. Both are fixed by adding `aria-label` or a properly associated `<label htmlFor>` to every affected control, re-verified afterwards by re-running axe-core against the same pages with zero remaining violations of either rule.

What was found and left open. A systemic colour-contrast defect remains: the muted secondary-text utility classes used throughout the interface (`text-black/40`, `text-black/50`, and their dark-mode equivalents) render at approximately 2.8:1 and 3.9:1 contrast against their backgrounds, both below the 4.5:1 WCAG AA minimum for normal text, and the violation recurs everywhere that styling is used — context labels, template names, helper text, the character counter. This is a design-system-level colour decision, not a single mistaken class, and changing it well past a quick patch means picking new opacity or colour values and checking them across every surface that uses them, in both themes; that is scoped

as a specific, prioritised item in Chapter 6 rather than patched inline here. The public share page — the one surface a recipient with no account ever sees — had zero violations of any kind, which is the one page where an accessibility defect would be hardest for the owner to notice on the recipient's behalf, and is corroborating rather than complete evidence, since no manual keyboard-navigation check has yet been performed on it or on the editor's fields.

Read plainly: the target was not met on first measurement, two real defects were found and fixed, one real and more consequential defect (contrast) was found and is scoped, not fixed, and the manual keyboard-navigation pass this section previously deferred is still deferred. This is a materially stronger position than "not yet audited," but it is not a pass.

5.7 Critical Evaluation Against Objectives

The preceding sections report what was tested and how; this section weighs that evidence against the objectives in §3.1 directly, rather than restating that the project "mostly succeeded." The evidence is not of even quality across objectives, and that unevenness — not any single missing feature — is the central finding of this chapter.

Security (FR14, and the security NFR). This is the objective with the strongest evidence behind it, and for a specific reason: it is the only one that was checked adversarially, against a claim the project had already made twice and could have simply repeated a third time. That check found two genuine, exploitable defects (§5.2 and the unauthenticated endpoint in §5.3) — the second found only because writing the regression tests meant reading every route file again, not because it was being looked for. Both are fixed, and both are now regression-tested (§5.3), so a route added next month that reintroduces the same mistake fails CI rather than shipping unnoticed. The objective is met and, for the first time in this project, mechanically protected against silent regression rather than merely fixed once.

Core API and sharing mechanics (FR7–FR11). Verified two ways now rather than one: manually end-to-end (§5.1), which is real evidence the feature works as designed under normal use but not that it is correct under an input the developer did not think to try; and, for the ownership and lifecycle boundaries specifically, by the route-level tests in §5.3, which do check exactly the kind of case a single manual walkthrough would not — a second user's token, a body field that disagrees with the session. The remaining gap is scope, not method: the new tests cover ownership and lifecycle transitions, not every field-validation and edge-case path the API exposes.

Usability (the usability NFR, and the claim in §1.5 that the system suits people with "no particular technical background"). The heuristic evaluation in §5.5 is genuine work and found three real defects, which is more than restating a plan would have produced. But it remains, structurally, the developer auditing the developer's own interface against a checklist, which cannot surface the confusions an actual outsider would hit and the developer would not think to look for. No one outside this project has ever used it. The claim in §1.5 is therefore still an assertion resting on the builder's own judgement, not a claim that has been tested against the population it names, and should be read that way rather than as a target substantially met.

Accessibility (the WCAG 2.1 AA target (W3C, 2018) in §3.1). This objective now has real evidence behind it, and the evidence is mixed rather than absent: an automated axe-core pass (§5.6) found and led to fixing two genuine defects — an unlabelled dropdown and six placeholder-only fields — and separately found, without yet fixing, a systemic colour-contrast defect affecting muted text everywhere it is used. This is a meaningfully stronger position than "unverified," because a tool independent of the developer's

own judgement is what found all three defects, not developer self-assessment. It remains short of the target in two respects: the contrast defect is real and open, and no manual keyboard-navigation pass has been done, so what an automated tool cannot check — focus order, screen-reader phrasing, anything requiring a human to operate the page without a mouse — is still exactly as unverified as before.

Data protection (FR17). Deletion was exercised manually and behaved correctly on the paths checked (§5.1), but was not tested against concurrent access — a share or edit submitted mid-deletion — which is exactly the kind of case a developer running through the happy path once is least likely to think to try.

Read across these five rows rather than in isolation, the pattern shifts once accessibility is no longer purely self-assessed: this project still verified best the objective it could check with an adversarial reading of its own code (security), and an automated, developer-independent tool now gives accessibility real, if incomplete, evidence too. Usability is the one objective still resting entirely on the developer's own judgement, because a heuristic evaluation is inherently that — no tool or database check can substitute for an outsider's confusion, which is exactly what a real participant study would surface and what this project has not obtained. That is not a criticism of effort; the heuristic evaluation, the accessibility pass, and the security audit were all done carefully. It does mean the project's claims about security and, now, about specific accessibility defects rest on meaningfully stronger evidence than its claim to be usable by "people with no particular technical background" (§1.5) does. A fair overall verdict is therefore mixed rather than nearly-complete: the project demonstrates that the core mechanism works, is not trivially broken under adversarial scrutiny, and has had two real accessibility defects found and fixed rather than merely claimed away — but it has not yet demonstrated, to anyone but its own author, that it is usable, and one known accessibility defect remains open. Chapter 6 should be read with that remaining asymmetry in mind, not as a short list of independent, equally-weighted loose ends.

Chapter 6: Conclusion

6.1 Summary

ContextID addresses a problem that is ordinary rather than exotic: most digital platforms force a single, permanent identity onto people who naturally, and legitimately, present themselves differently across the different spheres of their life. The delivered system separates identity into distinct, context-scoped cards; makes sharing an act that is revocable and auditable rather than final; extends that proposition with a choice of visual presentation per share and a recipient-facing contact-export and QR flow; and, since the draft report, replaced its interface's ad hoc modal handling with a consistent, accessibility-oriented component system, closed a real data-integrity gap in its input validation, and corrected a data-model requirement (a mandatory family name) that contradicted this report's own naming literature. Chapter 2 grounded this in Nissenbaum's contextual integrity and Cameron's Laws of Identity, and, in this revision, set the design against the contrasting self-sovereign-identity paradigm and the usable-security literature, rather than treating "privacy" and "security" each as a single undifferentiated goal. Chapter 5 evaluated the resulting system critically against that grounding: two real access-control defects and two real accessibility defects were found and fixed, a route-level test suite and an independent CI gate now exist where none did before, and what still falls short of the plan — a systemic colour-contrast issue, no independent usability evidence — is stated as exactly that rather than smoothed over.

6.2 Reflections

The most significant lesson of this project is procedural rather than technical: a systematic, adversarial re-reading of one's own code, done deliberately rather than assumed unnecessary because "the pattern is used elsewhere in the file," found a genuine vulnerability that two prior rounds of self-reported evaluation had missed. The same method, applied again while writing the regression tests meant to close that gap, found a second, unrelated one — an unauthenticated endpoint leaking user records — that no amount of re-reading the original bug would have surfaced, because it was a different bug in a different file. Applied to the interface rather than the API, the same discipline found three usability defects and, via an automated tool rather than manual reading, two accessibility defects. The pattern across all four evaluation areas is the same: every real defect this project found was found by checking a specific, falsifiable claim against the running system, and every prior round of self-reported evaluation that had skipped that step had missed something. The test suite and CI gate that were the clearest indicator of under-allocated effort in the draft and the previous revision now exist and pass; the equivalent indicator now is the absence of any evaluation evidence that came from someone other than the developer, which automated tools cannot supply no matter how many are run.

More broadly, the project illustrates a tension that recurs throughout the identity and privacy literature reviewed in Chapter 2: the gap between the *technical* mechanics of revocation — a token that stops resolving — and the *informational* reality that a recipient who has already viewed or copied a card retains it regardless. Comparing ContextID against the self-sovereign-identity paradigm in §2.4 sharpens this further: neither a centrally revocable link nor a holder-controlled verifiable credential closes this gap, because they trade the same underlying limitation for different guarantees in opposite directions. What a system in this space can do, and what ContextID does do, is make the technical act of withdrawal immediate, free, and available by default — a meaningful improvement over an emailed vCard or a printed card, though not the complete solution the word "revocable" might suggest at first reading. Naming that limitation clearly, rather than implying the problem is solved, is itself part of this report's intellectual contribution.

6.3 Future Work

Ordered by priority for the remaining project time, rather than by ambition:

1. **A formal usability study with independent participants** (§5.5) — now the single highest-priority gap, since it is the one evaluation area with no evidence beyond the developer's own judgement, and the heuristic evaluation already done is not a substitute for it.
2. **Fix the colour-contrast defect found in §5.6** — a scoped, well-understood fix (new opacity or colour values for the muted-text utility classes, checked across both themes) rather than an open-ended audit, now that the accessibility pass has identified exactly what needs to change.
3. **A manual keyboard-navigation and screen-reader pass** over the identity editor and public share page (§5.6), to check what an automated tool structurally cannot: focus order and screen-reader phrasing.
4. **Field-level, rather than whole-card, disclosure** — allowing a share to include only the fields a specific recipient needs, bringing the system closer to Cameron's Law of Minimal Disclosure (§2.3) than the current all-or-nothing model.
5. **Rate limiting on the public share endpoint, expiry-notification emails, and client-side date validation** — smaller, already-scoped items carried over from the prototype report.
6. **Multi-language name rendering via HTTP content negotiation** (§2.7), using **Accept-Language** to select among alternate name representations where a user has provided more than one.

6.4 Closing Statement

The system built so far demonstrates that contextual, revocable identity sharing is a buildable feature of an ordinary web application, using standard tools and imposing no special software or account requirement on the people receiving a shared identity. Its security and accessibility claims now rest on evidence beyond the developer's own assertion — a tested, CI-gated ownership model and an automated accessibility pass that has already found and fixed real defects — but its usability claim, per §1.5 and §5.7, does not yet. What remains is not a question of feasibility but of verification: closing the one evaluation gap that automated tooling cannot close by itself, which is exactly the work now prioritised for the remainder of the project.

References

- Allen, C. (2016) *The Path to Self-Sovereign Identity*. Life With Alacrity. Available at: <http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>
- Cameron, K. (2005) *The Laws of Identity*. Microsoft Corporation. Available at: <https://www.identityblog.com/stories/2005/05/13/TheLawsOfIdentity.pdf>
- Cavoukian, A. (2009) *Privacy by Design: The 7 Foundational Principles*. Information and Privacy Commissioner of Ontario, Canada.
- CM3035 Advanced Web Design. Course materials on secure account management and REST API design. University of London.
- Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD dissertation. University of California, Irvine. Available at: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Ishida, R. (2011) *Personal names around the world*. W3C Internationalization. Available at: <https://www.w3.org/International/questions/qa-personal-names>
- ISO/IEC 24760-1:2019. *Information technology — Security techniques — A framework for identity management — Part 1: Terminology and concepts*. International Organization for Standardization.
- Klensin, J. (2008) *RFC 5321: Simple Mail Transfer Protocol*. Internet Engineering Task Force. Available at: <https://www.rfc-editor.org/rfc/rfc5321>
- Marwick, A.E. and boyd, d. (2011) 'I Tweet Honestly, I Tweet Passionately: Twitter Users, Context Collapse, and the Imagined Audience', *New Media & Society*, 13(1), pp. 114–133.
- MDN Web Docs (2024) *Content negotiation*. Mozilla. Available at: https://developer.mozilla.org/en-US/docs/Web/HTTP/Content_negotiation
- MUI (2024) *Base UI*. Available at: <https://base-ui.com/>
- Nielsen, J. (1994) *Usability Engineering*. San Francisco: Morgan Kaufmann.
- Nissenbaum, H. (2004) 'Privacy as Contextual Integrity', *Washington Law Review*, 79(1), pp. 119–157.
- OWASP Foundation (2021) *OWASP Application Security Verification Standard (ASVS) 4.0*. Available at: <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Foundation (2023) *OWASP API Security Top 10*. Available at: <https://owasp.org/www-project-api-security/>
- Perreault, S. (2011) *RFC 6350: vCard Format Specification*. Internet Engineering Task Force. Available at: <https://www.rfc-editor.org/rfc/rfc6350>

- Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation).
- W3C (2018) *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation. Available at: <https://www.w3.org/TR/WCAG21/>
- W3C (2022) *Verifiable Credentials Data Model v1.1*. W3C Recommendation. Available at: <https://www.w3.org/TR/vc-data-model/>
- Whitten, A. and Tygar, J.D. (1999) 'Why Johnny Can't Encrypt: A Usability Evaluation of PGP 5.0', *Proceedings of the 8th USENIX Security Symposium*.